# ROS 2: Nodes, Topics, Services and Actions

## Contents


1. Overview . . . . . 2
2. Nodes and topics: message streams that no node waits for . . . . . 4
   - 2.1 One program per job: nodes . . . . . 4
   - 2.2 Topics: publish once, every subscriber gets a copy . . . . . 4
   - 2.3 Messages: typed data on a topic . . . . . 5
3. Services: one request, one response . . . . . 7
4. Actions: long jobs with feedback . . . . . 8
5. Parameters: one program, many robots . . . . . 10
6. Bags and RViz: recording, replaying and viewing . . . . . 11
7. Quality of service: when the network loses a message . . . . . 12
   - 7.1 Reliability: resend or drop a lost message . . . . . 12
   - 7.2 Durability and history: late joiners and queues . . . . . 14
8. Launch files: the whole stack in one command . . . . . 15
9. Lifecycle nodes: starting in the right order . . . . . 16
10. ROS 2 in Nav2, Autoware and Apollo . . . . . 17
11. Summary . . . . . 19
12. Sources . . . . . 20
13. Key terms . . . . . 21

**Where this fits:**

- **Builds on:** Layered autonomy stack and its rates.
- **Leads to:** Behaviour trees and the tick; Nav2 bring-up and the lifecycle manager.

## 1. Overview

**Key point:** ROS 2 runs a robot's software as separate programs, nodes, that pass typed messages: streams on topics, sent without waiting for the receivers, one-off questions on services, and long jobs with feedback on actions. Parameters, launch files, quality-of-service settings and lifecycle states configure, start and order the nodes; bags record their messages and RViz draws them.

The autonomy stack (G-3210) of the room robot now has its layers and their rates: a scan every 0.1 s, a local plan every 0.1 s, wheel commands every 0.05 s, and a route made once. The layers are separate programs, often written by different people and sometimes run on different computers. Nothing yet lets them exchange data. In the cycle at 2.4 s, when the person steps in:

- the scan of 682 readings that shows the person must reach two programs, the localizer and the local planner;
- the local planner's new plan must reach the controller, which sends a command every 0.05 s whether or not a new plan has come;
- the controller's command must reach the program that drives the wheels.

Each second the robot passes about 60 such messages: 10 scans, 20 wheel readings, 10 plans and 20 commands (notebook). If the controller had to call the planner and wait for its answer, it would tick at the planner's rate, the one-loop failure of sense–plan–act (G-3211). Communication also has other patterns: the map is a question asked once, and driving to the door is a job that takes 8.0 s. Figure 1 shows the room robot's programs and what passes between them.

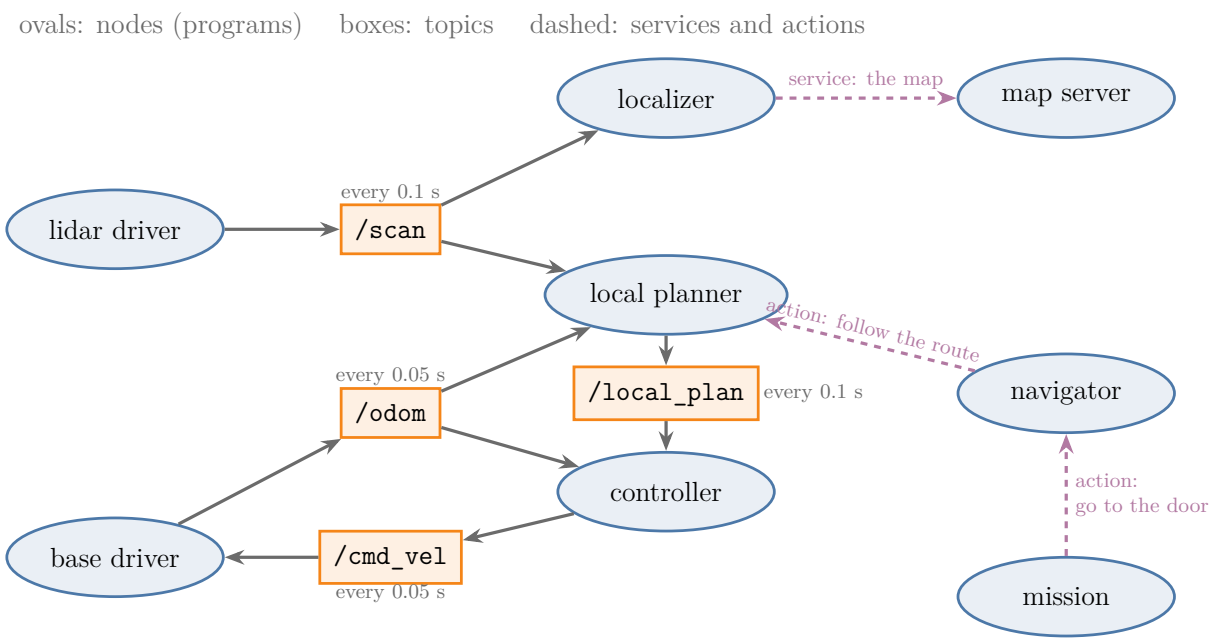



**Figure 1:** The room robot's stack as ROS 2 nodes. Ovals are nodes, the programs: lidar driver, base driver, localizer, local planner, controller, map server, navigator and mission. Boxes are topics, named streams of messages: /scan every 0.1 s from the lidar driver to the localizer and the local planner; /odom every 0.05 s from the base driver to the local planner and the controller; /local_plan every 0.1 s from the local planner to the controller; /cmd_vel every 0.05 s from the controller to the base driver. Dashed: the localizer asks the map server for the map (a service); the mission sends the navigator the goal and the navigator has the local planner follow the route (actions)

The opening question of this Note: **how do separate programs that run at different rates pass streams, questions and long jobs to each other without waiting, and how are they configured, started in order, recorded and watched?** The answer is ROS 2, built up in the order of the problems:

- programs and the streams between them: nodes, topics and messages (Section 2);
- a question asked once: services (Section 3);
- a long job with progress reports: actions (Section 4);
- one program for many robots: parameters (Section 5);
- replaying the 2.4 s moment and seeing it: bags and RViz (Section 6);
- messages lost on the way, and programs that start late: quality of service (Section 7);
- starting everything at once: launch files (Section 8), and in the right order: lifecycle nodes (Section 9);
- the same ideas in Nav2, Autoware and Apollo (Section 10).

## 2. Nodes and topics: message streams that no node waits for

> **Key point:** Each job runs in its own node. A node publishes messages to a named topic, and every node subscribed to that topic gets a copy; the publisher does not wait for the subscribers, and each subscriber uses the newest message it has, so nodes at 10 Hz and 20 Hz work together.

### 2.1 One program per job: nodes

The opening's first problem is passing the 2.4 s scan between separate programs. ROS 2 starts by making each program a named participant in the system. Each job of the stack runs as its own program: one drives the laser, one plans, one controls. A participant in a running ROS 2 system, usually one program doing one logical job, that sends and receives data through ROS 2, is a **node** (G-3222) (ROS 2 docs, Nodes). Nodes can run in the same process, in different processes, or on different machines; they find each other automatically when they start, by announcing themselves on the network (ROS 2 docs, Discovery).

**ROS 2** (G-3221), the Robot Operating System 2, is the open-source set of libraries and tools that lets such nodes talk, configures and starts them, and records and shows what they do. Its nodes talk through a communication layer called DDS, the Data Distribution Service (ROS 2 design, QoS). In Figure 1 the room robot has eight nodes. A node is the unit that can be replaced: a different laser needs only a different lidar driver, and the planner is unchanged.

### 2.2 Topics: publish once, every subscriber gets a copy

At 2.4 s the lidar driver has a new scan, and two nodes need it. Calling each of them and waiting for each to finish would tie the driver to the slowest of them. Instead the driver hands the scan to a named channel, `/scan`, and returns to reading the laser. Any node that has asked for `/scan` receives its own copy.

A named channel of messages that any node can send to or read from is a **topic** (G-3223); a node that sends on a topic is a **publisher**, and one that reads from it is a **subscriber** (G-3225) (ROS 2 docs, Topics). A topic can have zero or more publishers and zero or more subscribers, and neither side needs to know who the other is. The picture is a video channel: the channel posts a new video, everyone subscribed is sent it, and the channel never waits for its viewers.

Figure 2 plays the room robot's topics from 2.30 to 2.50 s, two frames per moment: published, then delivered. Watch the frames at 2.40 s: the scan with the person goes to both its subscribers, the local planner then publishes a bent plan, and the controller's next command follows it. Watch also 2.35 and 2.45 s, when the controller sends a command with no new plan.

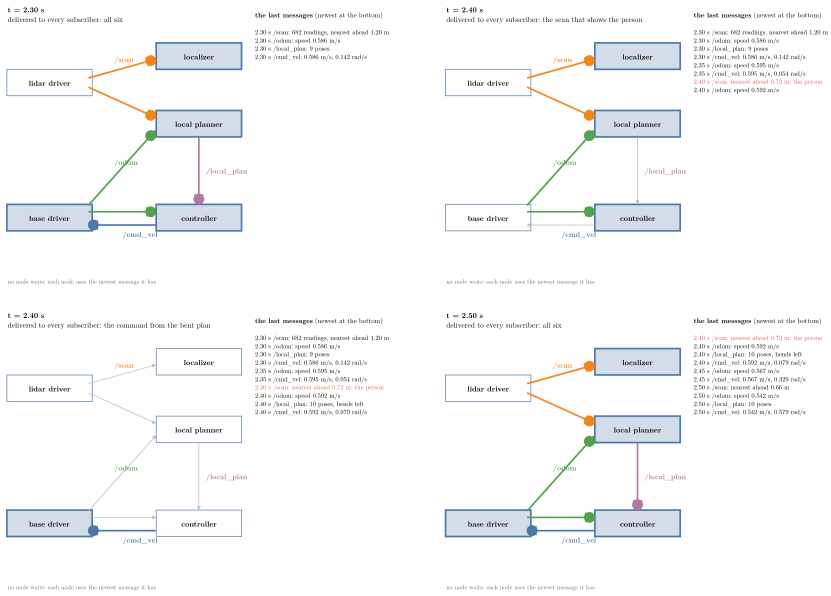



**Figure 2:** Animation: the room robot's nodes and topics from 2.30 to 2.50 s, two frames per moment: a message published (dot halfway along its arrow, publisher lit) and then delivered (dot at the end, subscribers lit). /scan (orange) every 0.1 s from the lidar driver to the localizer and the local planner; /odom (green) every 0.05 s from the base driver to the local planner and the controller; /local_plan (purple) every 0.1 s; /cmd_vel (blue) every 0.05 s from the controller to the base driver. The log on the right lists the last messages: at 2.40 s the scan's nearest reading ahead is 0.72 m, the person; then a 10-pose plan that bends left; then the command 0.592 m/s and 0.079 rad/s. At 2.45 s the controller sends 0.567 m/s and 0.329 rad/s from the plan it already has

The controller sends 20 commands a second and the planner 10 plans a second, and neither waits: at 2.45 s the controller uses the plan it received at 2.40 s (notebook). Working from the newest message is what the layered stack needed: each layer works from the newest output of the layer above. A new subscriber does not interrupt the flow of data to the others, so a recorder or a viewer can subscribe while the robot runs (ROS 2 docs, Topics).

### 2.3 Messages: typed data on a topic

The 2.4 s scan travels as a block of data whose layout every node agrees on. The data sent on a topic, with a fixed list of named, typed fields defined in a `.msg` file, is a **message** (G-3224) (ROS 2 docs, Topics). The scan uses the standard type `sensor_msgs/msg/LaserScan` (sensor_msgs/LaserScan). Its fields, with the 2.4 s scan of a scanner like the one in the coverage figure:

- `header`: the time the scan was taken, 2.4 s, and the coordinate frame its numbers are measured in, the laser's own;
- `angle_min` and `angle_max`: the first and last beam's direction, −119.86° and 119.86°, in radians in the message;
- `angle_increment`: the step between beams, 0.352°;

- `range_min` and `range_max`: the shortest and longest distance the scanner reports, 0.02 and 5.6 m;
- `ranges`: the 682 distances, one per beam, in metres.

The nearest of the 2.4 s ranges within 30° of straight ahead is 0.715 m, at −6.5°: the person (notebook). The fields are typed: `ranges` is a list of 32-bit numbers, and the code that builds or reads a message checks it. The units are agreed too: ranges in metres, angles in radians. Every node that reads `/scan` can therefore read the scan without asking the driver. The header's frame matters because the same reading lands at a different place on the map for every robot pose; the coordinate frame (G-2281) of each message is named in it, and ROS 2 keeps the poses of all frames relative to each other in a transform tree, so a node can move a reading from the laser's frame to the map's.

**Python:** a publisher and a subscriber, as in the ROS 2 tutorials, written with `rclpy`, ROS 2's Python library (not run in the notebook, which needs no ROS installation).

```python
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

class Controller(Node):
    def __init__(self):
        super().__init__('controller')
        # topic name, queue depth
        self.pub = self.create_publisher(Twist, 'cmd_vel', 10)
        # call tick every 0.05 s
        self.create_timer(0.05, self.tick)
    def tick(self):
        cmd = Twist()
        cmd.linear.x = 0.592    # m/s
        cmd.angular.z = 0.079   # rad/s
        self.pub.publish(cmd)   # returns at once

class Base(Node):
    def __init__(self):
        super().__init__('base_driver')
        self.create_subscription(Twist, 'cmd_vel', self.on_cmd, 10)
    def on_cmd(self, msg):      # called for each message
        print(msg.linear.x, msg.angular.z)
```

Topics carry streams well. They fit a question asked once poorly: a node that needs the map once would have to listen to a stream and pick out one message. The next section gives questions their own pattern.

## 3. Services: one request, one response

> **Key point:** A service is a request and one reply between two nodes: the client sends a request and waits, the server computes and replies. A service fits short questions such as "send me the map"; long jobs belong to actions.

The opening's second communication pattern is a question asked once. When the localizer starts, it needs the room's map one time, not a stream of maps every 0.1 s. It needs a call that is answered once: it asks the map server, waits and receives the map.

A pair of messages between two nodes (programs), one request and one reply, defined in a `.srv` file, is a **service** (G-3226): the node that answers is the service server, the node that asks is the service client (ROS 2 docs, Services). There is one server per service name and any number of clients. The everyday picture is a counter: we ask, wait, and are handed the answer.

Figure 3 plays the localizer asking the map server for the map. Nav2's map server offers exactly this service, `map_server/map`, of type `nav_msgs/srv/GetMap`: an empty request and a reply holding the map (Nav2 map_server.cpp). Watch the localizer's box during steps 2 to 4.

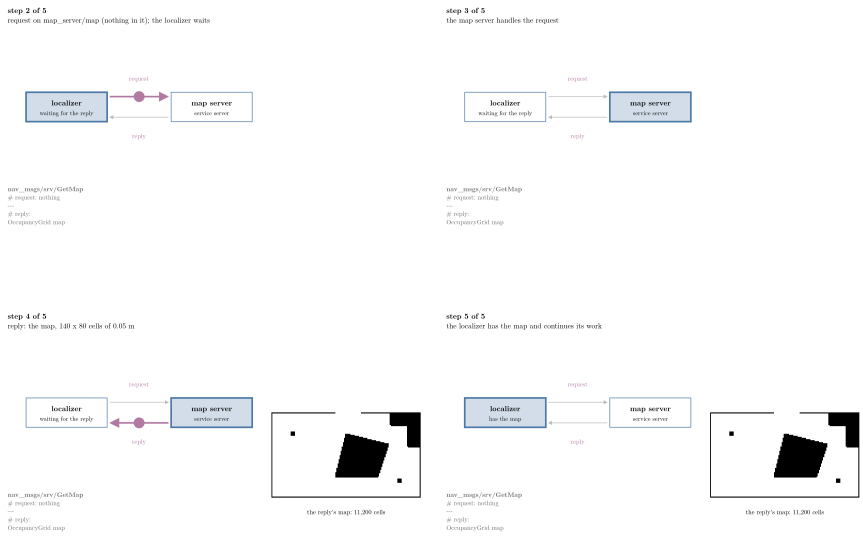



**Figure 3:** Animation: one service call, five steps. 1: the localizer starts and needs the map once. 2: it sends a request on map_server/map, which holds nothing, and waits. 3: the map server handles the request. 4: the reply carries the map, 140 × 80 cells of 0.05 m. 5: the localizer has the map and continues its work; on the right, the reply's map, black where a cell holds a wall, the table, the cabinet or a pole. The service type, nav_msgs/srv/GetMap, is printed below

The reply's map has 140 × 80 cells, one number each (notebook):

$$7.0/0.05 = 140$$

$$4.0/0.05 = 80$$

$$140 \times 80 = 11\,200 \text{ cells}$$

During steps 2 to 4 the localizer waits. Services are therefore meant to return quickly, and the ROS 2 documentation warns against using them for long-running work, especially work that may need to be stopped part way (ROS 2 docs, Services). Driving to the door takes 8.0 s; a service would block its caller for the whole drive, with no report of progress.

## 4. Actions: long jobs with feedback

> **Key point:** An action is a long job between two nodes: the client sends a goal, the server reports feedback while it works and a result when it ends, and the client can cancel. Driving to the door is one: the goal is the door pose, the feedback the distance left, the result succeeded after 8.0 s.

The opening's third communication pattern is a long job. The mission node's goal is the door pose, (3.6, 3.5) facing 90°. The drive takes 8.0 s, the person steps in on the way, and the mission node needs to know how far along the robot is and to be able to cancel the job.

A request for a long job, defined in an `.action` file, in which the client sends a goal, the server sends feedback while it works and a result when it ends, and the client may cancel, is an **action** (G-3227): the node doing the job is the action server, the node asking is the action client (ROS 2 docs, Actions). The everyday picture is ordering a meal for delivery: the order is the goal, the tracking updates are the feedback, the meal at the door is the result, and the order can still be cancelled.

Nav2's navigator offers this action as `NavigateToPose` (NavigateToPose.action):

- **goal:** a pose, the door pose (3.6, 3.5) facing 90°, and optionally which behaviour tree to run, the tree of task steps the navigator checks repeatedly (behaviour trees come in a later Note);
- **feedback:** the current pose, the time so far, the distance remaining along the plan, the estimated time remaining, the number of recoveries, and how far the robot is from its plan;
- **result:** an error code, 0 when the drive succeeded, and an error message.

Figure 4 follows the drive. Nav2 sends feedback at every tick of its behaviour tree, every 0.01 s (navigate_to_pose.cpp; nav2_params.yaml), 800 messages over the 8.0 s; the animation shows one a second and the moment the person steps in. Watch the distance remaining shrink and the goal stay the same at 2.4 s.

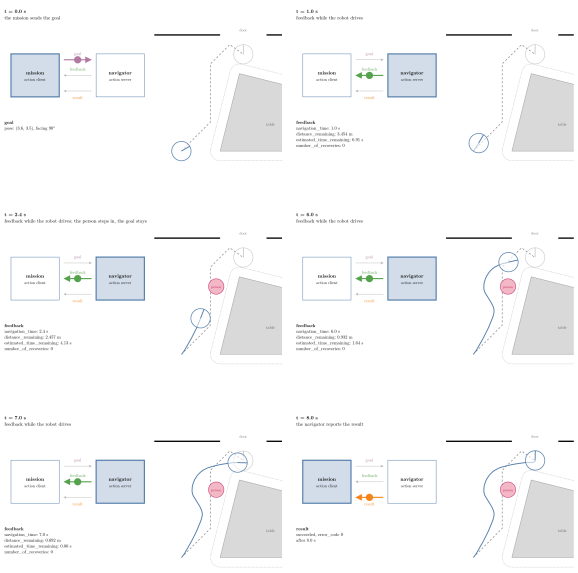


**Figure 4:** Animation: the navigate-to-pose action. Left: the mission (action client) and the navigator (action server) joined by three channels: goal, feedback, result. Right: the room, the plan from the robot to the door pose (grey dashed) and the robot. 0.0 s: the goal, the door pose (3.6, 3.5) facing 90°, is sent and accepted. Then feedback once a second: at 1.0 s the distance remaining is 3.454 m and the estimated time remaining 6.91 s; at 2.4 s, as the person steps in, 2.457 m; at 7.0 s 0.092 m. At 8.0 s the result: succeeded, error code 0

The distance remaining is measured along the plan the navigator follows: the A* route, 3.414 m between cell centres, with its ends moved to the robot and to the door pose, which makes it 3.698 m long (notebook). The estimated time remaining is that distance divided by the robot's present speed; at 1.0 s:

$$3.454/0.5 = 6.91 \text{ s}$$

Nav2 computes the estimate only when the robot moves faster than 0.01 m/s and more than 0.1 m is left (navigate_to_pose.cpp), which is why it reads 0 at 7.0 s, with 0.092 m to go.

Internally, an action is built from the two patterns already met: three services, to send the goal, to cancel it and to get the result, and two topics, for the feedback and the goal's status (ROS 2 design, Actions). A goal moves through the states accepted, executing and, at the end, succeeded, aborted or canceled. Section 2's stream, Section 3's question and this section's long job are the three ways ROS 2 nodes talk; the guideline is: topics for continuous data, services for quick calls, actions for anything that moves the robot or runs for a long time (ROS 2 docs, Topics vs Services vs Actions). How the nodes are set up for a particular robot is next.

## 5. Parameters: one program, many robots

> **Key point:** A parameter is a named, typed setting of a node, read at start-up from a file or the command line and changeable while the node runs, so the same program serves robots with different numbers.

The controller of Figure 2 was written for one robot, and our robot has its own numbers: each wheel at most 0.6 m/s. Nav2's controller is written for many robots. Writing those numbers into the code would mean one program per robot and a rebuild for every change. Instead the numbers are kept outside the code: a node's settings are like the inputs of a function, where the function is the node, the program.

A named setting of a node, such as `controller_frequency`, with a fixed type (a number, a string, a list) that is read when the node starts and can be read or changed while it runs, is a **parameter** (G-3228) (ROS 2 docs, Parameters). A node declares every parameter it accepts, with its type, so the name and type of each setting are fixed when the node starts and a wrong setting is less likely. Settings usually come from a YAML file. Figure 5 reads the same controller with two files.

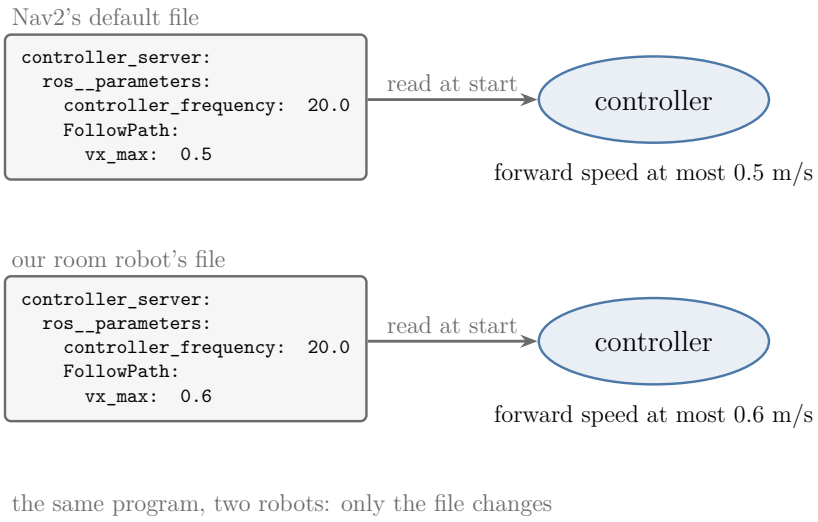



**Figure 5:** One program, two robots: the same controller node read with two parameter files. Nav2's default file sets controller_frequency to 20.0 and the controller's top forward speed, FollowPath vx_max, to 0.5 m/s; our room robot's file sets the same top speed to 0.6 m/s. Only the file changes

Nav2's default file runs the controller at 20 Hz (`controller_frequency: 20.0`) and caps its forward speed at 0.5 m/s (`vx_max: 0.5`) (nav2_params.yaml); our robot's file raises the cap to its own 0.6 m/s. The command `ros2 param set` asks a running node to change a parameter; the node may accept or refuse the change (ROS 2 docs, Parameters). With topics, services, actions and parameters the stack runs; the next problem is understanding a run after it is over.

## 6. Bags and RViz: recording, replaying and viewing

> **Key point:** A bag records the messages of chosen topics with their times, and playing it publishes them again on the same topics, so a moment such as 2.4 s can be replayed exactly; RViz subscribes to topics and draws them, so we can see what the robot sees.

The opening's last problem: the person steps in once, at 2.4 s. To check what the local planner did with that scan, or to try a new planner on it, we need the same scans again, at the same times, as often as needed.

A file that stores every message of chosen topics with the time it was sent, made by `ros2 bag record` and played back by `ros2 bag play`, is a **bag** (G-3229) (ROS 2 docs, Recording and playing back data). Recording is one more subscriber, a node reading the topics, so it does not interrupt the flow of data to the other nodes (ROS 2 docs, Topics); playing is one more publisher, on the same topic names, so the nodes downstream receive the messages as they came from the robot. Figure 6 records 0.2 s of the drive and plays it back; watch the 2.40 s scan go into the file, and come out again at 2.40 s on replay.

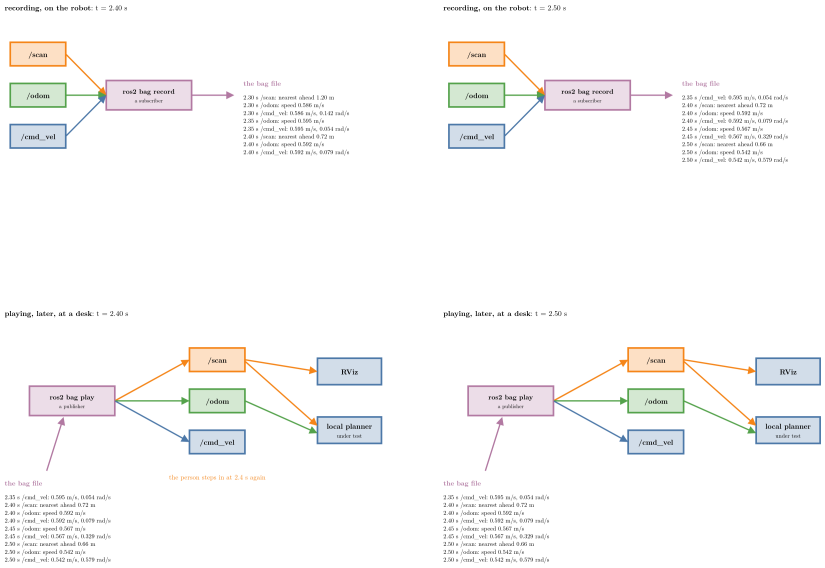



**Figure 6:** Animation: recording a bag, then playing it. Recording, on the robot, from 2.30 to 2.50 s: ros2 bag record, a subscriber, takes /scan, /odom and /cmd_vel and writes each message with its time into the bag file; the 2.40 s scan holds the person, nearest reading ahead 0.72 m. Playing, later, at a desk: ros2 bag play, a publisher, sends the stored messages on the same topics at the same times, where RViz and a local planner under test subscribe, so the person steps in at 2.4 s on every replay

Seeing the data matters as much as storing it: 682 numbers a scan do not show a person. The visualization tool of ROS that subscribes to topics such as the map, the scans, the plans and the robot's model and draws them in one 3D view is **RViz** (G-3230) (rviz README). Figure 7 is a sketch of what it would draw for our robot at 2.4 s.

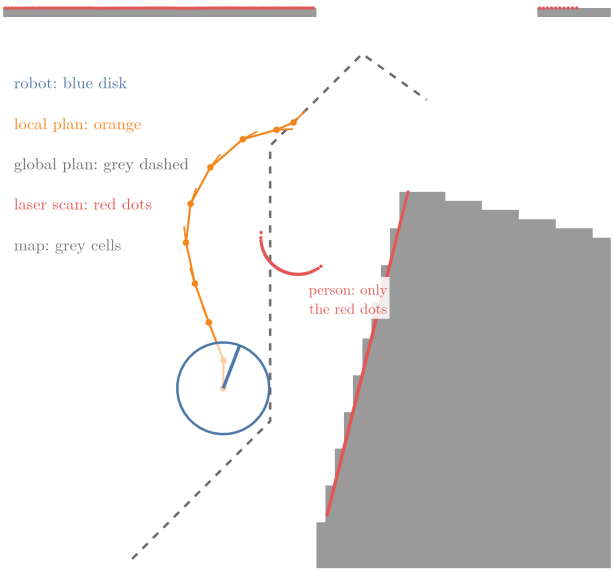



**Figure 7:** A sketch of what RViz draws for the room robot at 2.4 s, made from the same data the topics carry: the map (grey cells), the laser scan (red dots, the nearest straight ahead 0.72 m away), the global plan (grey dashed), the local plan (orange, bent left round the person) and the robot (blue disk). The person is not in the map: the robot knows of them only through the red dots

In Figure 7 the person is an arc of red dots on no grey cell: the scan detects what the map lacks, which is why the local planner bends. A bag of the drive replays this view at will. The stack can now run, be configured, recorded and seen. Two problems remain: messages lost on the network, and the order of start-up.

## 7. Quality of service: when the network loses a message

> **Key point:** Quality-of-service settings set how a topic behaves when the network loses messages or a node joins late: reliability (resend lost messages, or not), durability (keep the last message for nodes that join late, or not) and history (how many messages to queue). For sensor data the newest reading matters most, so scans use best effort; a map published once uses transient local.

### 7.1 Reliability: resend or drop a lost message

Suppose the robot's laser sits on one computer and the planner on another, joined by Wi-Fi, and the 2.4 s scan is lost on the link. Should the network send it again? ROS 1 sent its

messages over TCP, a network protocol that resends every lost packet and is unsuited to lossy links such as Wi-Fi; ROS 2 runs over UDP, a protocol that does not resend, through DDS, and lets each topic choose anything from as reliable as TCP to as best-effort as UDP (ROS 2 design, QoS; ROS 2 docs, Quality of Service settings).

A set of rules a publisher and a subscriber, the node that sends and the node that reads, agree on for delivering a topic's messages is the topic's **quality of service** (QoS) (G-3231) (ROS 2 docs, Quality of Service settings). The rule for lost messages is its **reliability** (G-3232): with *reliable*, a lost message is sent again, and newer messages wait until it arrives, because reliable delivery keeps the order; with *best effort*, a lost message is not resent and the next one is delivered as soon as it arrives (ROS 2 design, QoS). Figure 8's top half plays both for the lost 2.4 s scan. We assume the resent copy arrives 0.15 s late, at 2.55 s; the real delay depends on the link and the middleware. Watch the planner's newest scan at 2.5 s.




**Figure 8:** Animation, top: the lidar driver sends a scan every 0.1 s from 2.2 s; the 2.4 s scan is lost on the link. Best effort: the planner never gets 2.4, and gets 2.5 at 2.5 s. Reliable: 2.4 is sent again and arrives at 2.55 s, and 2.5 is held back until then. At 2.50 s the planner's newest scan is from 2.5 s with best effort but from 2.3 s with reliable. Bottom: durability. The map server publishes /map once, at 0 s; a node that starts at 5 s gets nothing with volatile durability, and the stored map at once with transient local

At 2.50 s the best-effort planner has the 2.5 s scan, taken that moment; the reliable planner still has the 2.3 s scan, because the 2.5 s scan waits behind the missing 2.4 s copy (notebook). The person is in both the 2.4 and the 2.5 scans, so the best-effort robot sees them at 2.5 s and the reliable one only at 2.55 s. For sensor data the newest reading matters more than getting every reading, which is why ROS 2's sensor-data profile is best effort with a short queue of 5 (rmw qos_profiles.h; ROS 2 docs, Quality of Service settings).

### 7.2 Durability and history: late joiners and queues

The map is the opposite case. The map server publishes it once. A node that starts later, such as RViz opened five seconds after the robot, would never receive it. The rule for messages sent before a subscriber joined is the topic's **durability** (G-3233): with *volatile*, nothing is kept for late joiners; with *transient local*, the publisher keeps its last messages and hands them to any subscriber that joins later (ROS 2 docs, Quality of Service settings). Figure 8's bottom half plays a node joining at 5 s.

The third setting matters when a subscriber falls behind. Suppose the planner is busy from 2.4 to 2.7 s while scans keep arriving every 0.1 s: three scans, from 2.5, 2.6 and 2.7 s, are waiting when it is free. How many waiting messages are kept is the topic's **history and depth** (G-3234): *keep last* with a depth, such as the last 10 messages, or *keep all* (ROS 2 design, QoS). Figure 9 compares a depth of 1 with keeping all.

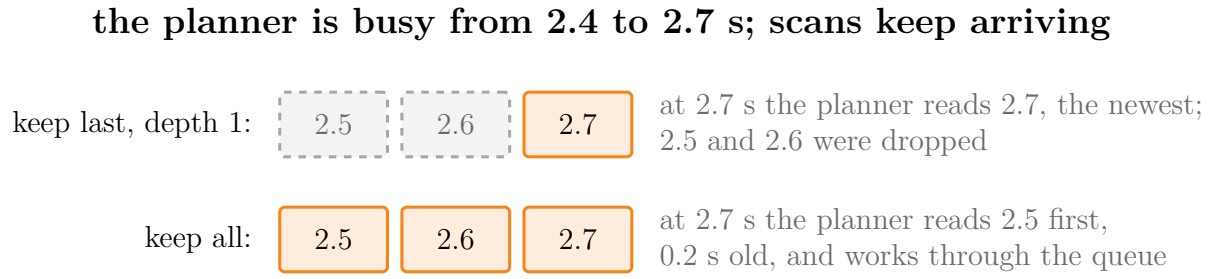



**Figure 9:** History and depth. The planner is busy from 2.4 to 2.7 s while scans keep arriving. Keep last with a depth of 1: only the 2.7 s scan is kept, so at 2.7 s the planner reads the newest scan; 2.5 and 2.6 were dropped. Keep all: all three are kept, so at 2.7 s the planner reads the 2.5 s scan first, 0.2 s old, and works through the queue

A short queue keeps a slow subscriber on recent data; a long one makes it work through old readings first.

Nav2 sets all three through profiles of its own (Nav2 qos_profiles.hpp; Nav2 map_server.cpp); its sensor-data profile keeps 10 messages, where ROS 2's keeps 5:

| Nav2 profile | Reliability | Durability | Depth | Meant for |
|---|---|---|---|---|
| sensor data | best effort | volatile | 10 | sensor streams such as scans |
| standard topic | reliable | volatile | 10 | ordinary topics |
| latched publisher | reliable | transient local | 1 | the map from the map server |

One pitfall: a publisher and a subscriber connect only if what the subscriber asks for is no stricter than what the publisher offers. A best-effort publisher and a reliable subscriber never

connect, so no messages flow between them (ROS 2 docs, Quality of Service settings). A viewer that asks for reliable scans from a best-effort laser driver shows nothing. Messages now arrive as each topic needs. What remains is starting many nodes, each with its file, in the right order.

## 8. Launch files: the whole stack in one command

> **Key point:** A launch file lists the nodes to start, with their parameters and names, so one command starts the whole stack and one Ctrl+C stops it; launch files can include other launch files.

Our stack has eight nodes, programs (Figure 1), and, from Section 9, a lifecycle manager; Nav2's default bring-up has 17 managed nodes and a lifecycle manager, each with its parameter file. Starting each by hand in its own terminal is slow and error-prone. A file, written in Python, XML or YAML, that describes which nodes to start, with which parameters, names and arguments, and that `ros2 launch` runs as a whole, is a **launch file** (G-3235) (ROS 2 docs, Launch). `ros2 launch` also watches the processes it started, and on Ctrl+C it shuts them all down (ROS 2 docs, Launch). Figure 10 starts the room robot.

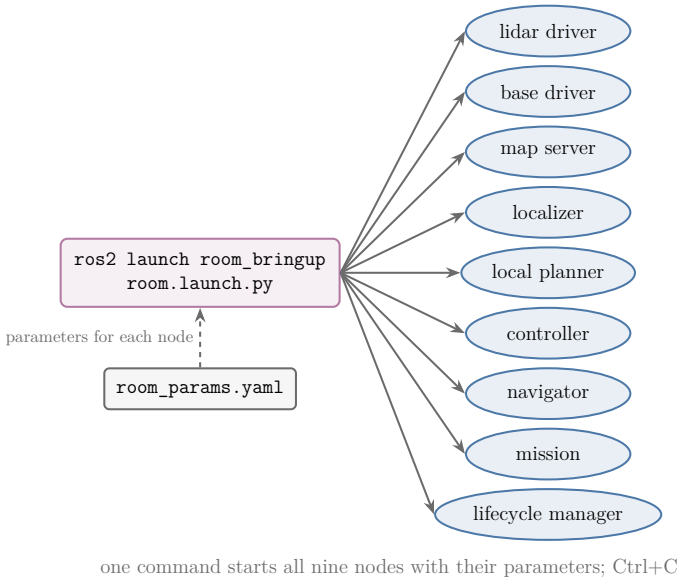



**Figure 10:** One launch file, room.launch.py, started with ros2 launch, starts all nine nodes of the room robot: lidar driver, base driver, map server, localizer, local planner, controller, navigator, mission and lifecycle manager, each with its parameters from room_params.yaml; Ctrl+C stops them all

> **Python:** the start of such a launch file. `ros2 launch` calls `generate_launch_description` and starts every node it returns.
>
> ```
> from launch import LaunchDescription
> from launch_ros.actions import Node
> ```

```
def generate_launch_description():
    p = ['room_params.yaml']
    return LaunchDescription([
        Node(package='room_drivers',
             executable='lidar_driver', parameters=p),
        Node(package='room_drivers',
             executable='base_driver', parameters=p),
        Node(package='room_nav',
             executable='controller', parameters=p),
        # ... the other six nodes
    ])
```

A launch file starts the nodes at the same moment, in no particular order. The controller could then send commands before the map server has loaded the map or the localizer knows where the robot is. Starting in order needs the nodes' cooperation.

## 9. Lifecycle nodes: starting in the right order

> **Key point:** A lifecycle node moves through set states, Unconfigured, Inactive, Active and Finalized, only when told to, so a manager can configure every node, then activate them in order, and shut them down in reverse; Nav2 starts its 17 nodes this way.

The launch file of Section 8 starts every node at once, but the controller must not drive before the map and the localizer are ready. A node that does its work as soon as it is created cannot be held back. ROS 2 therefore gives nodes states that change only on request.

A node with four main states, Unconfigured (just created), Inactive (configured: parameters read, publishers made, but processing nothing), Active (working) and Finalized (about to be destroyed), that moves between them only through the transitions configure, activate, deactivate, cleanup and shutdown requested by another program, is a **lifecycle node** (G-3236), also called a managed node (ROS 2 design, Node lifecycle). The purpose is control from outside: start-up tools can make sure every node is set up correctly before any of them acts, and a node can be restarted or replaced while the system runs (ROS 2 design, Node lifecycle). An inactive node answers no requests: Nav2's map server ignores a map request unless it is Active (Nav2 map_server.cpp).

Nav2 runs its servers as lifecycle nodes, and its lifecycle manager, a node that holds the list of managed nodes, starts them: it configures every node in the list in order, then activates every node in order, and on shut-down deactivates, cleans up and shuts them down in reverse order (Nav2 lifecycle manager README; lifecycle_manager.cpp). Figure 11 plays the start-up and shut-down for Nav2's default list of 17 nodes, the controller server's state lit on the left.

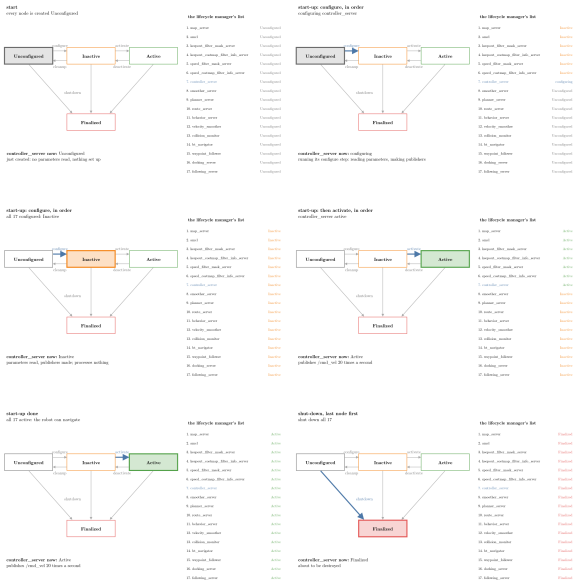


**Figure 11:** Animation: left, the lifecycle state machine of one node, the controller server, its current state lit: Unconfigured, Inactive, Active, Finalized, joined by configure, cleanup, activate, deactivate and shutdown. Right: the lifecycle manager's list, Nav2's default 17 nodes, map_server and amcl first, then the four zone-filter servers. Start: all Unconfigured. Configure in order (map_server, amcl, the four filter servers, controller_server, ...): all Inactive. Then activate in order: all Active, and the controller publishes /cmd_vel 20 times a second. Shut-down, last node first: deactivate, clean up, shut down: all Finalized

The order is what matters: by the time the controller is activated, the map server and the localizer, first in the list, are already active. While the system runs, the manager keeps a heartbeat with each active node, one every 0.25 s by default, and declares a node lost after 4.0 s without one (lifecycle_manager.cpp). The room robot's stack can now be started, ordered, configured, connected and recorded; the last section looks at how the real stacks do it.

## 10. ROS 2 in Nav2, Autoware and Apollo

> **Key point:** Nav2 and Autoware are built from ROS 2 nodes started by nested launch files, Nav2's under one lifecycle manager; Apollo uses its own framework, Cyber RT, whose components, channels, services, launch files and record files play the parts of ROS 2's nodes, topics, services, launch files and bags.

The patterns of this Note are how the real stacks are built (Figure 12):

- **Nav2** (`ros-navigation/navigation2`, main). `nav2_bringup/launch/bringup_launch.py` includes `localization_launch.py`, which starts the map server and AMCL, Nav2's particle-filter localizer; `keepout_zone_launch.py` and `speed_zone_launch.py`, switched on by default, which start two servers each for the costmap's keep-out and speed-limit zones; and `navigation_launch.py`, which starts 11 servers: controller,

smoother, planner, route and behavior servers, the velocity smoother, the collision monitor, the BT navigator, the waypoint follower, and the docking and following servers. `bringup_launch.py` then starts one lifecycle manager whose list holds those 17 lifecycle nodes. Its arguments `autostart` and `use_composition` choose whether the manager starts them at once and whether the nodes share one process. The parameters come from `nav2_bringup/params/nav2_params.yaml`.

- **Autoware** (`autowarefoundation/autoware_launch`, main). `autoware_launch/launch/autoware.` includes one launch file per component, vehicle, system, map, sensing, localization, perception, planning and control, each behind a switch such as `launch_planning`.
- **Apollo** (`ApolloAuto/apollo`, master). Apollo runs on its own framework, Cyber RT, in the repository's `cyber/` folder. Its modules are components that talk through channels with readers and writers, offer services and parameters, are wired by DAG files, started by launch files, and recorded in record files (Cyber RT terms). The control module, for example, is a timer component that runs every 10 ms (`modules/control/control_component/dag/control.dag`).

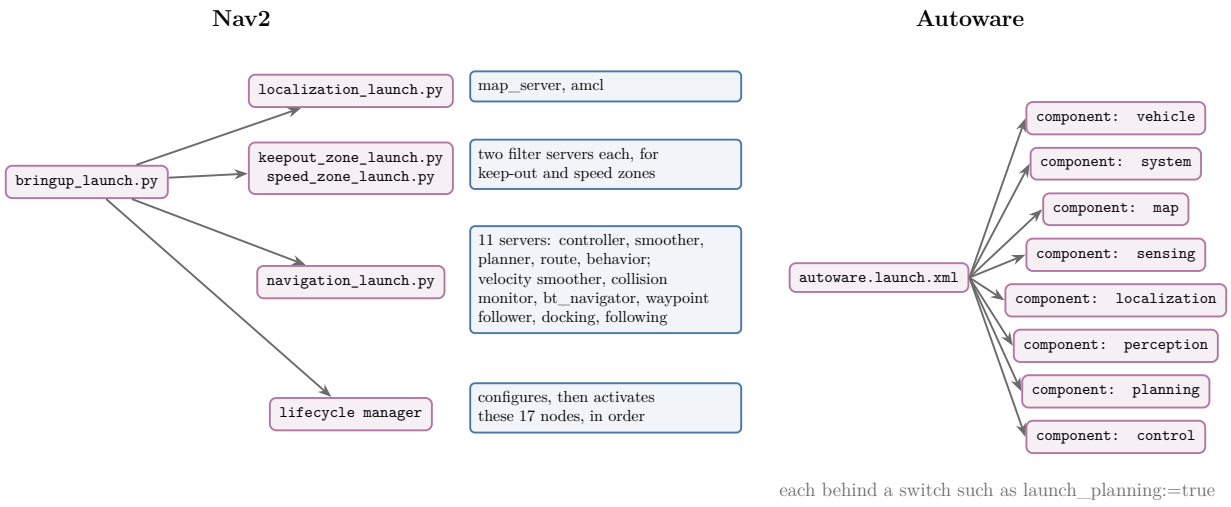



**Figure 12:** How Nav2 and Autoware start. Nav2: bringup_launch.py includes localization_launch.py (map_server, amcl), keepout_zone_launch.py and speed_zone_launch.py (two filter servers each) and navigation_launch.py (11 servers: controller, smoother, planner, route, behavior; velocity smoother, collision monitor, bt_navigator, waypoint follower, docking, following), and starts a lifecycle manager that configures, then activates these 17 nodes in order. Autoware: autoware.launch.xml includes one launch file per component (vehicle, system, map, sensing, localization, perception, planning, control), each behind a switch such as launch_planning:=true

## 11. Summary

| ROS 2 piece | What it carries or does | In the room | Why |
|---|---|---|---|
| node | one job, one program | eight nodes | each part can be replaced alone |
| topic | a stream, publish and subscribe | /scan, /odom, /local_plan, /cmd_vel | no node waits; 10 Hz and 20 Hz nodes work together |
| service | one request, one reply | the map, 11,200 cells | a question asked once |
| action | goal, feedback, result, cancel | to the door in 8.0 s | long jobs need progress reports and a way to cancel |
| parameter | a typed setting | top speed 0.6 m/s | one program, many robots |
| bag and RViz | record, replay, draw | the 2.4 s scan again | a moment can be studied |
| QoS | reliability, durability, history | scans best effort, map transient local | the newest scan matters; late joiners need the map |
| launch file | start everything | one command, eight nodes | no hand-started terminals |
| lifecycle node | states on request | 17 Nav2 nodes, in order | nothing drives before the map is ready |

- The opening's programs pass the 2.4 s scan to both its subscribers and the bent plan to the controller within the cycle, because ROS 2's **topics** (G-3223) let a **publisher** hand each message to every **subscriber** (G-3225) without waiting, so the controller at 20 Hz uses the newest plan from the planner at 10 Hz.
- **ROS 2** (G-3221) runs the stack as **nodes** (G-3222), one per job, which find each other on start-up, so a new laser needs only a new driver.
- A **message** (G-3224) has typed, named fields in agreed units, so every node can read the 682-reading scan without asking its sender.
- A **service** (G-3226) is one request and one reply, so the localizer gets the 11,200-cell map once; its caller waits, so services are kept short.
- An **action** (G-3227) has a goal, feedback and a result and can be cancelled, so the 8.0 s drive to the door reports 3.454 m to go at 1 s and succeeds at 8.0 s.
- A **parameter** (G-3228) is a typed setting read at start-up, so Nav2's controller runs our robot at 0.6 m/s instead of its default 0.5 m/s without new code.
- A **bag** (G-3229) records and replays topics with their times and **RViz** (G-3230) draws them, because a moment such as the person stepping in happens only once.
- **Quality of service** (G-3231) sets **reliability** (G-3232), **durability** (G-3233) and **history and depth** (G-3234), so scans use best effort (the reliable planner saw the

person 0.05 s later) and the map uses transient local (a late node still gets it).
- A **launch file** (G-3235) starts all the nodes with their parameters at once, and **lifecycle nodes** (G-3236) let a manager configure and then activate them in order, so the controller never drives before the map and the localizer are ready.

The room robot's stack can now run as a working ROS 2 system. One part is still missing: testing it safely before it meets a real person, in a simulator that steps the physics of the robot, its wheels and the floor, which is the next Note, physics simulators.

## 12. Sources

### Built from

- Raymond Andrade, "What Is ROS2? - Framework Overview", YouTube, https://www.youtube.com/watch?v=7TVWlADXwRw: nodes, publisher and subscriber on a topic with a second subscriber, the video-channel picture, a service with request and response, an action with goal, feedback and result, a wheel-radius parameter, and bag files
- Kevin Wood | Robotics & AI, "Learn ROS 2: Beginner to Advanced Course (Concepts and Code)", YouTube, https://www.youtube.com/watch?v=HJAE5Pk8Nyw: parameters as the inputs of a node, launch files instead of one terminal per node, RViz as ROS's visualization tool
- ROS 2 documentation, source files in https://github.com/ros2/ros2_documentation (rolling, read 2026-10-12): `ROS-Framework/About-Nodes.rst` (ROS 2 docs, Nodes), `nodes/About-Discovery.rst` (ROS 2 docs, Discovery), `interfaces/About-Topics.rst` (ROS 2 docs, Topics), `interfaces/About-Services.rst` (ROS 2 docs, Services), `interfaces/About-Actions.rst` (ROS 2 docs, Actions), `interfaces/Topics-Services-Actions.r` (ROS 2 docs, Topics vs Services vs Actions), `About-Parameters.rst` (ROS 2 docs, Parameters), `interfaces/topics/About-Quality-of-Service-Settings.rst` (ROS 2 docs, Quality of Service settings), `Developer-Tools/About-Launch.rst` (ROS 2 docs, Launch), `Recording-And-Playing-Back-Data.rst` (ROS 2 docs, Recording and playing back data)
- ROS 2 design articles, https://design.ros2.org: "ROS 2 Quality of Service policies" (ROS 2 design, QoS), "Actions" (ROS 2 design, Actions), "Managed nodes" (ROS 2 design, Node lifecycle)
- ROS 2 code, https://github.com/ros2: `rviz/README.md` (rviz README); `rmw/include/rmw/qos_profiles.h` (rmw qos_profiles.h); `common_interfaces`: `sensor_msgs/msg/LaserScan.msg` (sensor_msgs/LaserScan), `geometry_msgs/msg/Twist.msg`, `nav_msgs/srv/GetMap.srv`
- Nav2, the navigation2 repository on GitHub, https://github.com/ros-navigation/navigation2 (main, read 2026-10-12): `nav2_bringup/params/nav2_params.yaml` (nav2_params.yaml); `nav2_bringup/launch/bringup_launch.py`, `localization_launch.py`, `keepout_zone_launch.py` `speed_zone_launch.py` and `navigation_launch.py`; `nav2_lifecycle_manager/README.md` (Nav2 lifecycle manager README) and `src/lifecycle_manager.cpp` (lifecycle_manager.cpp); `nav2_map_server/src/map_server/map_server.cpp` (Nav2 map_server.cpp); `nav2_ros_common/include/nav2_ros_common/qos_profiles.hpp`

(Nav2 qos_profiles.hpp); `nav2_msgs/action/NavigateToPose.action` (NavigateToPose.action); `nav2_bt_navigator/src/navigators/navigate_to_pose.cpp` (navigate_to_pose.cpp)
- Autoware, the launch repository on GitHub, https://github.com/autowarefoundation/autoware_launch (main, read 2026-10-12): `autoware_launch/launch/autoware.launch.xml`
- Apollo, the apollo repository on GitHub, https://github.com/ApolloAuto/apollo (master, read 2026-10-12): `docs/04_CyberRT/CyberRT_Terms.md` (Cyber RT terms) and `modules/control/control_component/dag/control.dag`

**Other references**

- ROS 2 documentation, "Managing node lifecycles - example", `ROS-Framework/nodes/Working-with-n` in https://github.com/ros2/ros2_documentation: a common use of lifecycle nodes is hardware such as lidars and motor drivers that must be started, configured and shut down in a controlled order

## 13. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

| Term | Meaning |
|---|---|
| Node (ROS 2) (G-3222) | A participant in a running ROS 2 system, usually one program doing one logical job such as driving the laser or planning, that sends and receives data through topics, services and actions; nodes find each other on start-up, so one can be replaced without touching the others. |
| ROS 2 (Robot Operating System 2) (G-3221) | An open-source set of libraries and tools that runs a robot's software as separate programs (nodes) passing typed messages over topics, services and actions, and that configures, starts, records and shows them; it lets parts written by different people work together at their own rates. |
| Topic (G-3223) | A named channel of messages, such as `/scan`, that any node can publish to or subscribe to; every subscriber receives a copy of each message and the publisher does not wait, so it carries streams such as scans and commands between nodes running at different rates. |

| Term | Meaning |
|---|---|
| Publisher and subscriber (publish–subscribe) (G-3225) | A node's two roles on a topic: the publisher hands each message to the topic and goes on with its work, and every subscriber receives its own copy; neither needs to know the other, so many nodes can share a stream without waiting. |
| Message (ROS 2) (G-3224) | The data sent on a topic, with a fixed list of named, typed fields defined in a `.msg` file, such as a `LaserScan` with its angles and its list of ranges in metres; because the layout and units are agreed, any node can read it. |
| Service (ROS 2) (G-3226) | A request and one reply between two nodes, defined in a `.srv` file, such as asking the map server for the map: the client sends the request and waits, the server computes and replies; it suits short questions, not long jobs. |
| Action (ROS 2) (G-3227) | A long job between two nodes, defined in an `.action` file: the client sends a goal, the server sends feedback while it works and a result when it ends, and the client can cancel, such as driving to a door pose; it is built from three services and two topics. |
| Parameter (ROS 2) (G-3228) | A named, typed setting of a node, such as `controller_frequency: 20.0`, read at start-up from a file or the command line and changeable while the node runs; it lets one program serve robots with different numbers. |
| Bag (ROS 2) (G-3229) | A file that stores every message of chosen topics with the time it was sent, made by `ros2 bag record` and played back by `ros2 bag play` on the same topic names, so a moment of a run can be replayed and studied exactly. |
| RViz (G-3230) | ROS's visualization tool: it subscribes to topics such as the map, the scans, the plans and the robot's model and draws them together in one 3D view, so we can see what the robot sees and plans. |

| Term | Meaning |
|---|---|
| Quality of service (QoS) (G-3231) | The rules a publisher and a subscriber agree on for delivering a topic's messages, chiefly reliability, durability and history; they decide what happens when the network loses a message or a node joins late, and mismatched rules stop the two from connecting. |
| Reliability (QoS) (G-3232) | The quality-of-service rule for lost messages: reliable resends a lost message and keeps newer ones waiting until it arrives; best effort drops it and delivers the next at once, which suits sensor data, where the newest reading matters most. |
| Durability (QoS) (G-3233) | The quality-of-service rule for messages sent before a subscriber joined: volatile keeps none; transient local has the publisher keep its last messages and hand them to late joiners, so a map published once still reaches a node started later. |
| History and depth (QoS) (G-3234) | The quality-of-service rule for how many messages are queued: keep last with a depth, such as the last 10, or keep all; a short queue keeps a slow subscriber working on recent data instead of a growing backlog. |
| Launch file (G-3235) | A file, in Python, XML or YAML, that lists the nodes to start with their parameters, names and arguments, and that `ros2 launch` runs as a whole; one command starts the whole stack and one Ctrl+C stops it, and launch files can include others. |
| Lifecycle node (managed node) (G-3236) | A node with the states Unconfigured, Inactive, Active and Finalized that it changes only when another program requests a transition (configure, activate, deactivate, cleanup, shutdown); a manager can then set nodes up and start them in order, and restart or replace them while the system runs. |

| Term | Meaning |
|---|---|
| Autonomy stack (G-3210) | The whole of a robot's software arranged in layers, sensing at the bottom and mission, behaviour, motion and control above it, each layer running at its own rate and working from the latest output of the layer above; the slow layers plan far ahead while the fast ones react, so a new reading reaches the wheels in one fast cycle. |
| Sense–plan–act (deliberative control) (G-3211) | A way to run a robot in one fixed loop: read the sensors, update a model of the world and plan, then act on the plan, and repeat; the robot looks again only when the whole loop is done, so the slowest step, such as planning over the whole map, sets how often it can react. |
| Coordinate frame (G-2281) | A chosen origin and set of axes used to give positions as numbers; every position or pose is stated relative to some frame. |